

# Documentación de API y Arquitectura Cloud: Sistema de Gestión de Ambulancias (IMSGA)

## 1. Introducción y Propósito del Sistema

El **sistema de gestión de ambulancias (IMSGA)** es una plataforma móvil de alta disponibilidad diseñada para entornos médicos críticos. Su objetivo principal es resolver la fragmentación y manualidad del proceso de despacho y registro de atenciones médicas de emergencia. Al centralizar la información que antes se gestionaba de forma dispersa, el sistema elimina retrasos, previene errores de registro y garantiza una trazabilidad total de los casos atendidos.

El sistema funciona conectando en tiempo real dos perfiles clave de usuarios:

- **Personal de Control:** Encargados de la coordinación, recepción de llamadas de emergencia y despacho de unidades.
- **Personal de Ambulancia (Médicos, Enfermeros, Conductores):** Personal operativo responsable de la navegación y el registro clínico de las atenciones directamente en el terreno.

---

## 2. Stack Tecnológico y Componentes

Para garantizar un entorno robusto, seguro y escalable, el sistema se divide de forma desacoplada entre una aplicación móvil (Frontend) y un servidor central (Backend):

| Componente / Capa    | Tecnología Utilizada                       | Responsabilidad Operativa                                                     |
|----------------------|--------------------------------------------|-------------------------------------------------------------------------------|
| Presentación (Móvil) | React Native + Expo (SDK 52) & Expo Router | Interfaz de usuario multiplataforma, formularios y navegación en la app.      |
| Gestión de Estado    | React Context API                          | Manejo del estado global de la app por módulos individuales.                  |
| Lógica Backend       | Django + Django REST Framework (DRF)       | Procesamiento de lógica de negocio, validaciones y exposición de la API REST. |

| Componente / Capa               | Tecnología Utilizada             | Responsabilidad Operativa                                                                 |
|---------------------------------|----------------------------------|-------------------------------------------------------------------------------------------|
| <b>Base de Datos Relacional</b> | PostgreSQL                       | Persistencia y almacenamiento seguro de datos de usuarios, pacientes y bitácoras.         |
| <b>Almacenamiento Cloud</b>     | Amazon S3                        | Depósito seguro de documentos clínicos en formato JSON y sus respectivas firmas digitales |
| <b>Servidor y Despliegue</b>    | Ubuntu Server + Nginx + Gunicorn | Alojamiento en AWS EC2, proxy inverso para enrutamiento, manejo de CORS y servidor WSGI.  |

### 3. Arquitectura del Sistema y Patrones de Diseño

El sistema opera bajo una arquitectura **Cliente-Servidor desacoplada**. La comunicación se realiza mediante peticiones seguras HTTP/HTTPS y el intercambio de datos estructurados.

#### 3.1 Patrones Implementados en el Frontend (React Native)

- **Context + Provider & Repository Pattern:** Cada módulo (Autenticación, Despachos, Atenciones) posee su propio contexto. Estos actúan como repositorios aislados que encapsulan las llamadas HTTP de la API, ocultando la complejidad del código a las pantallas de la aplicación
- **Compound Components:** Los formularios complejos de atención médica y pacientes se componen de subcomponentes reutilizables que comparten de forma nativa el mismo contexto de validación (*react-hook-form*).
- **Adapter Pattern:** Funciones traductoras (*mappearControl*, *mappearWorker*) que transforman las respuestas crudas de la API al formato exacto que necesita la interfaz móvil, logrando que cambios en la base de datos no rompan la aplicación.
- **Guard Pattern:** Filtros de seguridad a nivel de código que verifican el rol del usuario antes de dibujar las pantallas en el celular, redirigiendo al inicio de sesión si no hay credenciales válidas.

#### 3.2 Patrones Implementados en el Backend (Django REST Framework)

- **Class-Based Views (CBV):** Vistas organizadas limpiamente en clases donde cada método HTTP (GET, POST, PATCH) resuelve una acción específica.
- **Serializer Pattern:** Capa encargada de recibir la información enviada por el teléfono, validarla rigurosamente contra las reglas del negocio y transformarla en registros aptos para la base de datos.
- **Transaction Pattern (`transaction.atomic()`):** Asegura que las operaciones críticas se completen al 100% o no se guarde nada (por ejemplo, al crear un

despacho y asignar una ambulancia al mismo tiempo), evitando datos corruptos o a medias si se corta la conexión.

---

## 4. Seguridad, Control de Acceso y Cumplimiento Legal

Dada la criticidad de un entorno médico, el sistema implementa una estricta política de seguridad basada en tres pilares:

### 4.1 Control de Acceso Basado en Roles (RBAC)

El sistema implementa un modelo de Control de Acceso Basado en Roles (RBAC) mapeado en la base de datos a través de la entidad `Role`. Los roles definidos fueron los siguientes:

1. **Control** : Personal administrativo con privilegios totales para la creación de despachos, programación de rutas, creación de , inicialización de códigos dinámicos MFA y administración de inventarios globales.
2. **Médico/enfermero**: Personal de salud a bordo de las unidades de emergencia habilitado exclusivamente para la lectura de sus despachos asignados, captura signos vitales, y cierres de las fichas de atención.
3. **Conductor / Chofer (chofer)**: Personal logístico con acceso a una pantalla mediante la cual cambia los estados de las ambulancias, para saber cual esta disponible o ocupada.

### 4.2 Flujo de Autenticación Segura y Doble Factor (MFA)

Tras la validación exitosa del doble factor de autenticación (Credenciales + Código Dinámico TOTP), la aplicación móvil realiza una petición complementaria obligatoria para asegurar la comunicación en tiempo real del ecosistema. El frontend registra el identificador de notificaciones push del dispositivo móvil mediante el endpoint `/api/token/post/`. Este proceso mapea de manera persistente el token único del dispositivo al usuario en sesión, garantizando que el centro de control pueda alertar instantáneamente a los celulares de las ambulancias en ruta sin intermediación manual.

### 4.3 Integridad del Registro Clínico y Auditoría (Cumplimiento Legal)

- **Firma Digital Inalterable**: Cuando el personal médico finaliza una atención, el backend compila la información en un archivo JSON, calcula un hash matemático único (SHA-256) y genera una firma digital criptográfica con el algoritmo ECDSA. Tanto el documento como el archivo de firma (`.sig`) se suben a Amazon S3 en un formato inmutable en base64.

- **Preservación de Auditoría Histórica:** Mediante el modelo `LogAuditoria`, cada acción médica o de despacho queda registrada con una marca de tiempo (*timestamp*) automática e inalterable. Para efectos legales, este registro preserva el RUT del operador de forma independiente, lo que permite realizar auditorías forenses completas incluso si el usuario ha sido desactivado del sistema en el futuro.
- 

## 5. Diccionario de Endpoints de la API

A continuación se detallan los accesos técnicos reales que utiliza la aplicación móvil para comunicarse con el servidor central:

### Módulo 1: Autenticación, Sesión y Control de Acceso

Este módulo se encarga del apretón de manos inicial, la mitigación de ataques CSRF y la verificación multifactor obligatoria (MFA).

#### 1.1 Obtener Token CSRF preliminar

- **Ruta:** `GET /api/login/`
- **Permisos:** Público (Cualquier dispositivo)
- **Descripción:** Petición necesaria previa al login. El servidor responde inyectando la cookie de seguridad y entregando las directivas iniciales para el envío de formularios seguros.
- **Códigos de Respuesta:**
  - `200 OK:` Éxito.
- **Estructura JSON de Respuesta:**

```
{
  "detail": "CSRF cookie set successfully."
}
```

#### 1.2 Inicio de Sesión Completo (Login con MFA)

- **Ruta:** `POST /api/login/`
- **Serializer Asociado:** `AuthenticationSerializer`
- **Permisos:** Público.
- **Descripción:** Valida la identidad del operador mediante tres factores. Si todo coincide, establece la cookie de sesión nativa cifrada y expone los datos del usuario adjuntando su rol operativo para modular las vistas de la app móvil.
- **Estructura JSON del Request (Body):**

```
{
  "username": "medico_perez",
  "password": "PasswordSegura123!",
  "totp_code": "482019"
}
```

- **Códigos de Respuesta:**
  - o 200 OK: Credenciales validadas con éxito. Se adjuntan cookies seguras en las cabeceras HTTP (Set-Cookie: sessionId=...; HttpOnly; Secure).
  - o 401 Unauthorized: Contraseña incorrecta o código TOTP de 6 dígitos inválido/expirado.
- **Estructura JSON de Respuesta (200 OK):**

```
{
  "id": 4,
  "username": "medico_perez",
  "first_name": "Carlos",
  "last_name": "Pérez",
  "rut": "12.345.678-9",
  "rol_nombre": "medico",
  "is_active": true
}
```

### 1.3 Registro/Renovación de Token de Notificaciones Push

- **Ruta:** POST /api/token/post/
- **Serializer Asociado:** DeviceToken
- **Permisos:** Cualquier usuario autenticado.
- **Descripción:** Enlaza el token único generado por el motor de Firebase Cloud Messaging (FCM) o Expo Push al usuario en sesión para el envío inmediato de despachos en terreno.
- **Estructura JSON del Request (Body):**

```
{
  "token": "ExponentPushToken[xxxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx]"
}
```

- **Códigos de Respuesta:**
  - o 200 OK / 201 Created: Token almacenado y enlazado.
  - o 400 Bad Request: Token vacío o con formato incorrecto.

## Módulo 2: Gestión de Personal, Roles y Equipos (Tripulaciones)

Reservado de forma estricta para el personal de **Control** (`ControlProfileOnly`).

### 2.1 Obtener Datos de un Funcionario

- **Ruta:** GET `/api/personal/`
- **Serializer Asociado:** `PersonalSerializer`
- **Descripción:** Obtiene los datos del trabajador autenticado que solicita la petición (`request.user`).
- **Estructura JSON de Respuesta (200 OK):**

```
{
  "id": 12,
  "username": "control_gomez",
  "first_name": "María",
  "last_name": "Gómez",
  "rut": "15.987.654-3",
  "rol_nombre": "control",
  "is_active": true
}
```

### 2.2 Registrar Nuevo Funcionario (Alta en Sistema)

- **Ruta:** POST `/api/personal/add/`
- **Descripción:** Registra un nuevo funcionario. El backend genera de inmediato una clave transitoria e inicializa el secreto TOTP de 32 caracteres para el MFA.
- **Códigos de Respuesta:**
  - o 201 Created: Operario añadido.
  - o 400 Bad Request: RUT inválido o duplicado en la base de datos.
- **Estructura JSON de Respuesta (201 Created):**

```
{
  "status": "Usuario creado con éxito",
  "username": "temporaltens",
  "temp_password": "InitialPassword_99",
  "totp_secret": "NBSWY3DPEB3W64TBNQXD2",
  "qr_uri": "otpauth://totp/imSystem:temporaltens?secret=NBSWY3DPEB3W64TBNQXD2&issu"
}
```

### 2.3 Listar Todos los Grupos (Tripulaciones Activas)

- **Ruta:** GET /api/grupo/
- **Serializer Asociado:** GrupoPersonalSerializer
- **Descripción:** Devuelve todos los grupos junto con la nómina detallada en tiempo real de quiénes van a bordo de la ambulancia física (filtrando los que tengan fecha\_salida=None).
- **Estructura JSON de Respuesta (200 OK):**

```
[
  {
    "id": 2,
    "nombre_grupo": "Tripulación Alpha - Ambulancia AMB-20",
    "miembros_activos": [
      {
        "id": 5,
        "first_name": "Juan",
        "last_name": "Rojas",
        "rut": "18.222.333-K",
        "rol": "medico"
      },
      {
        "id": 9,
        "first_name": "Luis",
        "last_name": "Contreras",
        "rut": "11.444.555-6",
        "rol": "chofer"
      }
    ]
  }
]
```

## 2.4 Creación de Grupos Base

- **Ruta:** POST /api/grupo/crear/
- **Serializer Asociado:** CrearGrupoSerializer
- **Descripción:** Instancia un nuevo equipo operativo unificado inyectando un arreglo de IDs correspondientes al personal que lo integrará.
- **Estructura JSON del Request (Body):**

```
{
  "nombre_grupo": "Tripulación Delta Nocturna",
  "personal": [5, 9, 14]
}
```

- **Códigos de Respuesta:**
  - 201 Created: Grupo instanciado y personal adscrito.
  - 400 Bad Request: Formato de lista vacío ("Debe asignar al menos 1 persona al grupo").

## 2.5 Vincular Funcionario a un Grupo (Subir a Ambulancia)

- **Ruta:** POST /api/grupo/suscribir/
- **Serializer Asociado:** AgregarMiembroGrupo
- **Descripción:** Registra de manera viva la incorporación de un profesional a un equipo de guardia.
- **Estructura JSON del Request (Body):**

```
{
  "grupo_id": 2,
  "personal_id": 7
}
```

## 2.6 Remover Funcionario de un Grupo (Baja de Ambulancia)

- **Ruta:** POST /api/grupo/desuscribir/
- **Serializer Asociado:** RemoverMiembroGrupo
- **Descripción:** En lugar de borrar la tupla, actualiza la propiedad `fecha_salida` con el timestamp de la petición actual, resguardando el registro histórico.
- **Estructura JSON del Request (Body):**

```
{
  "group_id": 2,
  "personal_id": 7
}
```

# Módulo 3: Gestión de Pacientes e Historiales Clínicos

## 3.1 Obtener Datos de Ficha por RUT

- **Ruta:** POST /api/pacientes/get/
- **Serializer Asociado:** ParamPacienteSerializer
- **Descripción:** Consulta la base de datos relacional para recuperar antecedentes básicos del paciente.
- **Estructura JSON del Request (Body):**



```
{  
  "rut": "12.345.678-9"  
}
```

- Estructura JSON de Respuesta (200 OK):

```
{  
  "rut": "12.345.678-9",  
  "nombre_completo": "Juan Carlos Bodoque",  
  "fecha_nacimiento": "1985-05-15",  
  "direccion": "Av. Alemania 410",  
  "comuna": "Valparaíso",  
  "telefono": "+56988887777",  
  "condicion_paciente": "Hipertenso crónico, operado de meniscos."  
}
```

### 3.2 Registro Clínico de Pacientes (Nueva Ficha)

- **Ruta:** POST /api/pacientes/add/
- **Serializer Asociado:** PacienteSerializer
- **Descripción:** Inicializa la ficha clínica que servirá de ancla histórica en el sistema.
- **Estructura JSON del Request (Body):**

```
{  
  "rut": "12.345.678-9",  
  "nombre_completo": "Juan Carlos Bodoque",  
  "fecha_nacimiento": "1985-05-15",  
  "direccion": "Av. Alemania 410",  
  "comuna": "Valparaíso",  
  "telefono": "+56988887777",  
  "condicion_paciente": "Hipertenso crónico"  
}
```

- **Códigos de Respuesta:**
  - 201 Created: Paciente registrado con éxito.
  - 400 Bad Request: Datos mal estructurados o RUT ya existente.

## Módulo 4: Gestión de Despachos y Emergencias Médicas

### 4.1 Crear Despacho Médico (Ingreso Inicial)

- **Ruta:** POST /api/despachos/add/
- **Serializer Asociado:** CreateDespachoSerializer
- **Permisos:** ControlProfileOnly (Personal de Control).
- **Descripción:** Registra un incidente de auxilio enlazando el RUT del paciente. El backend asigna automáticamente el estado inicial "recibido". Completa de forma automática el campo `creado_por` tomando las credenciales vigentes desde el `request.user`.
- **Estructura JSON del Request (Body):**

```
{
  "direccion_origen": "Plaza Aníbal Pinto, Valparaíso",
  "direccion_destino": "Hospital Carlos Van Buren",
  "descripcion_llamado": "Paciente masculino con fuerte dolor opresivo en el pecho",
  "paciente_rut": "12.345.678-9"
}
```

- **Estructura JSON de Respuesta (201 Created):**

```
{
  "id": 45,
  "estado": "recibido",
  "direccion_origen": "Plaza Aníbal Pinto, Valparaíso",
  "fecha_llamado": "2026-06-12T12:00:00Z"
}
```

## 4.2 Asignar Unidad y Activar Ruta

- **Ruta:** PATCH /api/despachos/asignar/
- **Serializer Asociado:** AsignarDespachoSerializer
- **Permisos:** ControlProfileOnly.
- **Descripción:** Vincula una ambulancia física y un grupo operativo humano específico al incidente. Modifica el estado del despacho a "asignado", guarda el timestamp de la asignación y de forma automática rellena el campo `asignado_por` mediante `request.user`. Esta acción dispara las notificaciones push en segundo plano a la tripulación asignada.
- **Estructura JSON del Request (Body):**

```
{
  "despacho_id": 45,
  "amb_id": 2,
  "grupo_id": 1,
  "is_emergency": true
}
```

- **Códigos de Respuesta:**
  - o 200 OK: Modificación exitosa, unidad en tránsito.

### 4.3 Programar Despacho Clínico Diferido

- **Ruta:** POST /api/despachos/programar/
- **Serializer Asociado:** ProgramarDespachoSerializer
- **Permisos:** ControlProfileOnly.
- **Descripción:** Se utiliza para agendar atenciones diferidas (por ejemplo, traslados médicos programados). Modifica el estado del registro a "programado".
- **Estructura JSON del Request (Body):**

```
{
  "despacho_id": 45,
  "fecha_programada": "2026-06-13T08:30:00Z",
  "grupo_id": 1
}
```

### 4.4 Ver Todos los Despachos del Sistema (Bitácora Global)

- **Ruta:** GET /api/despachos/getall/
- **Permisos:** ControlProfileOnly.
- **Descripción:** Devuelve un listado completo e histórico de todos los incidentes médicos registrados para control forense o monitoreo centralizado.
- **Códigos de Respuesta:**
  - o 200 OK: Retorna el arreglo completo de objetos de despacho.

### 4.5 Visualizar Despacho Específico Asignado (Uso en Terreno)

- **Ruta:** GET /api/despachos/get/
- **Permisos:** WorkerProfileOnly / Operarios a bordo.
- **Descripción:** El equipo de la ambulancia consume este endpoint para cargar en su pantalla móvil los datos geográficos de origen, la descripción del incidente y los antecedentes de la ficha clínica del paciente que van a asistir. El backend devuelve los datos exclusivos según el grupo del operario en sesión.
- **Estructura JSON de Respuesta (200 OK):**

```

{
  "despacho_id": 45,
  "estado": "asignado",
  "direccion_origen": "Plaza Aníbal Pinto, Valparaíso",
  "direccion_destino": "Hospital Carlos Van Buren",
  "descripcion_llamado": "Paciente masculino con fuerte dolor opresivo en el pecho",
  "paciente": {
    "nombre_completo": "Juan Carlos Bodoque",
    "edad": 41,
    "condicion_paciente": "Hipertenso crónico"
  }
}

```

## Módulo 5: Registro Clínico de Urgencia y Cierre del Evento

### 5.1 Registro de Atención Médica Completa (Cierre de Ficha)

- **Ruta:** POST /api/atenciones/add/
- **Serializer Asociado:** PayloadSerializer
- **Permisos:** WorkerProfileOnly (Médico, Enfermero o TENS a cargo).
- **Descripción: Endpoint Core del Negocio.** Al terminar el procedimiento en terreno, se envía un payload JSON masivo que contiene las capturas seriadas de signos vitales, la cronología radial de tiempos (QTH), el preinforme clínico y el descuento fino de insumos médicos ocupados de la ambulancia. El backend procesa todo en un bloque transaccional atómico (`transaction.atomic()`), cambia el estado del Despacho a "finalizado", compila el JSON clínico final, calcula su hash SHA-256 e implementa la firma digital criptográfica Ed25519, subiendo el documento e integridad de forma inmutable a AWS S3. El campo `rut_registrador` se vincula automáticamente con el usuario logueado en sesión.
- **Estructura JSON del Request (Body):**

```
{
  "despacho": {
    "despacho_id": 45,
    "ambulancia_id": 2,
    "hora_salida": "2026-06-12T12:05:00Z",
    "hora_llegada": "2026-06-12T12:45:00Z"
  },
  "signos_vitales": [
    {
      "presion_sistolica": 140,
      "presion_diastolica": 90,
      "frecuencia_cardiaca": 88,
      "saturacion_oxigeno": 96,
      "temperatura": "36.50",
      "fr": 18,
      "fio2": 21,
      "hgt": 110,
      "gcs": 15,
      "eva": 4,
      "hora": "1215",
      "observaciones": "Primer control estandarizado en QTH."
    }
  ]
}
```

```

],
"preinforme": {
  "pre_informe": "Se estabiliza a paciente en ruta mediante oxigenoterapia",
  "motivo_llamado": "Dolor torácico",
  "estado_paciente": "Estable dentro de su gravedad"
},
"cronologia": {
  "hora_llamada": "1200",
  "despacho_movil": "1203",
  "llegada_qth1": "1212",
  "salida_qth1": "1230",
  "llegada_qth2": "1245",
  "salida_qth2": "1315",
  "categoria": "C1"
},
"insumos_utilizados": [
  {
    "presentacion_id": 4,
    "cantidad_usada": 1,
    "observaciones": "Ampolla de aspirina administrada vía oral"
  }
],
"rut_receptor": "11.222.333-4"

```

- Estructura JSON de Respuesta (201 Created):

```

{
  "status": "Atención guardada y sellada con éxito",
  "atencion_id": 102,
  "despacho_estado": "finalizado",
  "estado_sello": "Registrada",
  "sello_electronico": "sha256:d8f76a5b4c3e2d10f9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1"
}

```

## 5.2 Retornar Información de Atención Específica (Lectura)

- **Ruta:** GET /api/atenciones/
- **Serializer Asociado:** ParamAtencionSerializer
- **Descripción:** Permite auditar una atención ya guardada y corroborar sus metadatos de firma en S3 y su estatus legal.
- **Estructura JSON del Request (Body):**

```
{  
  "id": 102  
}
```

## Módulo 6: Gestión Fina de Inventario e Insumos por Ambulancia

### 6.1 Obtener Catálogo General de Insumos Activos

- **Ruta:** GET /api/inv/
- **Descripción:** Lista todo el catálogo base del sistema segmentado por su categoría médica correspondiente.
- **Códigos de Respuesta:**
  - o 200 OK.

### 6.2 Carga Masiva de Stock a Ambulancia (Ingreso de Insumos)

- **Ruta:** POST /api/inv/add/
- **Serializer Asociado:** BulkAddInsumoSerializer
- **Permisos:** ControlProfileOnly.
- **Descripción:** Permite abastecer o reponer la farmacia interna de un vehículo de emergencia a través de un listado por lotes (Bulk).
- **Estructura JSON del Request (Body):**

```
{  
  "items": [  
    {  
      "nombre_insumo": "Aspirina",  
      "categoria_id": 1,  
      "cantidad": "200.0",  
      "unidad_medida_id": 2,  
      "stock": 50,  
      "ambulancia_id": 2  
    }  
  ]  
}
```

### 6.3 Transferencia/Movimiento de Insumos entre Unidades (Traspaso)

- **Ruta:** POST /api/inv/move/
- **Serializer Asociado:** MoveItemSerializer

- **Permisos:** `ControlProfileOnly`.
- **Descripción:** Modifica directamente y cuadra el inventario en caso de que una ambulancia le traspase recursos farmacológicos a otra de forma presencial en terreno.
- **Estructura JSON del Request (Body):**

```
{
  "presentacion_id": 4,
  "ambulancia_from_id": 2,
  "ambulancia_to_id": 5,
  "cantidad": 10
}
```

## 6.4 Actualizar Stock de Insumo Específico (Cuadratura Manual)

- **Ruta:** `POST /api/inv/update/`
- **Serializer Asociado:** `UpdateInsumoSerializer`
- **Permisos:** `ControlProfileOnly`.
- **Descripción:** Permite regularizar mermas, pérdidas o ingresos manuales corrigiendo los números exactos del casillero de la ambulancia.
- **Estructura JSON del Request (Body):**

```
{
  "presentacion_id": 4,
  "ambulancia_id": 2,
  "cantidad": 40
}
```

# Módulo 7: Interoperabilidad Internacional (Estándar FHIR)

## 7.1 Exportar Ficha Clínica en Estándar HL7 / FHIR

- **Ruta:** `GET /api/fhir/`
- **Permisos:** Usuarios autenticados del sistema de salud.
- **Descripción:** Transforma la atención médica guardada internamente a un recurso estructurado de salud interoperable internacional (`FHIR Bundle` - Recurso tipo *Patient / Observation*), facilitando la conexión inmediata con fichas clínicas institucionales de grandes centros médicos públicos o privados.
- **Códigos de Respuesta:**
  - 200 OK: Retorna el formato JSON normado bajo directivas HL7 FHIR.



## Módulo 8: Auditoría Interna del Sistema (Logs)

### 8.1 Listar Logs de Auditoría Forense

- **Ruta:** GET /api/logs/
- **Permisos:** Reservado con exclusividad al Administrador de Control.
- **Descripción:** Recupera la tabla histórica generada por el modelo LogAuditoria. Despliega el tipo de módulo (atencion, inventario, ambulancia, etc.), la descripción precisa y el RUT del operador de forma independiente para asegurar total transparencia, cumpliendo con las regulaciones de auditoría médica vigentes.
- **Estructura JSON de Respuesta (200 OK):**

```
[
  {
    "id": 1452,
    "tipo": "atencion",
    "rut_usuario": "18.222.333-K",
    "descripcion": "Cierre clínico y firmado criptográfico del despacho #45. Documen",
    "timestamp": "2026-06-12T13:16:00Z"
  }
]
```

### 8.2 Endpoint de Gestión de Unidades Móviles (Ambulancias)

- **Ruta:** GET /api/ambulancias/
- **Serializer Asociado:** AmbulanciaSerializer
- **Descripción:** Expone el estado actual de la flota de ambulancias y sus correspondientes estados de disponibilidad (disponible, en\_despacho, mantencion, fuera\_servicio).
- **Estructura JSON de Respuesta (200 OK):**

```
[
  {
    "id": 2,
    "patente": "KPDX-45",
    "modelo": "Mercedes-Benz Sprinter 315 CDI",
    "estado_disponibilidad": "disponible"
  }
]
```

## Módulo 9: Errores del sistema

### 9.1 Módulo de Autenticación y Control de Acceso (Errores)

- **9.1 Inicio de Sesión Completo (Login con MFA)**

- **Código de error:** 401 Unauthorized (Código TOTP inválido, expirado o credenciales de usuario incorrectas).
- **Estructura JSON de Error:**

```
{  
  "detail": "Código de autenticación multifactor (TOTP) inválido o expirado."  
}
```

- **9.2 Registro/Renovación de Token de Notificaciones Push**

- **Código de error:** 400 Bad Request (Token ausente o mal formateado por el motor del cliente móvil).
- **Estructura JSON de Error:**

```
{  
  "token": [  
    "Este campo no puede estar en blanco o el formato de cadena push es inválido."  
  ]  
}
```

### 9.2 Módulo de Gestión de Personal y Equipos (Errores)

- **9.1 Registrar Nuevo Funcionario (Alta en Sistema)**

- **Código de error:** 409 Conflict (El RUT ingresado ya existe en los registros indexados de la base de datos).
- **Estructura JSON de Error:**

```
{
  "rut": [
    "Ya existe un usuario registrado con este RUT en la plataforma imSystem."
  ]
}
```

## ● 9.2 Creación de Grupos Base

- **Código de error:** 400 Bad Request (Se envió el arreglo de IDs de personal vacío).
- **Estructura JSON de Error:**

```
{
  "personal": [
    "Debe asignar al menos 1 persona al grupo"
  ]
}
```

## 9.3 Módulo de Gestión de Pacientes e Historiales Clínicos (Errores)

### ● 9.1 Obtener Datos de Ficha por RUT

- **Código de error:** 404 Not Found (El paciente consultado no cuenta con historial ni registro previo en el sistema).
- **Estructura JSON de Error:**

```
{
  "detail": "No se encontró ningún paciente con el RUT especificado en la base de da"
}
```

## 9.4 Gestión Fina de Inventario e Insumos por Ambulancia (Errores)

### ● 9.1 Transferencia/Movimiento de Insumos entre Unidades (Traspaso)

- **Código de error:** 400 Bad Request (Se intentó traspasar una cantidad de insumos mayor al stock físico real disponible en la ambulancia de origen).
- **Estructura JSON de Error:**

```
{
  "cantidad": [
    "Operación inválida. La cantidad solicitada excede el stock físico registrado en
  ]
}
```

## 6. Aseguramiento de la Calidad: Matriz de Pruebas de la API

### Matriz Resumida de Control de Errores HTTP

- **400 Bad Request:** Parámetros faltantes, tipos de datos erróneos o fallas en las restricciones de los serializers (ej: Enviar una lista de personal vacía al crear un grupo).
- **401 Unauthorized:** No se han proveído credenciales válidas en las cookies o falló la verificación dinámica del código TOTP.
- **403 Forbidden:** El usuario está correctamente logueado, pero su rol operativo no cuenta con el nivel de acceso suficiente (ej: Un usuario con rol `medico` intentando poblar inventarios o crear un despacho).
- **409 Conflict:** Errores en la base de datos por llaves únicas (ej: Intentar registrar un Paciente o Personal con un RUT que ya existe).
- **500 Internal Server Error:** Excepciones no controladas en el servidor, caídas de la conexión con PostgreSQL o pérdida temporal de conexión con los servicios AWS S3.

---

## 7. Estado del Proyecto

A fecha de **Junio de 2026**, el núcleo del sistema IMSGA se encuentra operativo y desplegado en entornos de pruebas controlados en AWS EC2, con los módulos primordiales ya validados.